

Enabling Deep Learning at the Edge

Matthew Jones

Submitted for the Degree of Master of Science in
Machine Learning



Department of Computer Science
Royal Holloway University of London
Egham, Surrey TW20 0EX, UK

September 11, 2021

Redacted public copy. Personal details, the signature and the acknowledgements have been removed, and the appendix has been omitted. Spelling and grammar have been corrected.

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: 17,182

Student Name: Matthew Jones

Date of Submission: 12/09/2021

Abstract

Applications at the Edge of computing are traditionally constrained to devices with limited processing power, memory, and energy. Conversely, tasks for Deep learning such as training and inference are considered computationally expensive, power-hungry and require a large memory bandwidth. We explore ways in which Deep Neural Networks (DNNs) can be made to work under the constraints of Edge devices. We look at currently utilised techniques and create a Python library designed for reducing a fully trained Deep Neural Network's (DNN) size, processing and memory requirements, so they can be made to work on Edge devices. The library includes an implementation of the Winning Ticket Search algorithm [4] which we combine with quantisation techniques and study the effect on a variety of network architectures and deployments on a range of Edge devices. We conduct one of the first studies of the interplay between LTH(Lottery Ticket Hypothesis) and Quantisation Aware Training(QAT) and find that they are complementary and can produce DNNs with a significant reduction in complexity, which enables a wide range of new applications utilising DNNs on Edge devices.

Contents

1	Introduction	1
1.1	Challenges of the Project	1
1.2	Project Aims	2
1.3	Paper Layout	2
2	Background Research	4
2.1	Overview	4
2.2	Quantisation	4
2.2.1	Overview	4
2.2.2	Techniques	5
2.2.3	QAT (Quantisation Aware Training)	6
2.3	Pruning	7
2.3.1	Techniques	8
2.3.2	The Lottery Ticket Hypothesis (LTH)	8
3	Experiments	12
3.1	Overview	12
3.2	Datasets	13
3.3	Networks	13
3.4	Devices	15
3.5	Design and Implementation Choices	18
3.6	Experimental Method	19
3.7	Phase 1: Winning Ticket Search	19
3.8	Phase 2: Winning Ticket Search with QAT	20
3.9	Phase 3: Post Training Testing	21
4	Results	24
4.1	Dense Network	24
4.2	VGG-19	28
4.3	DLA	33
4.4	AlexNet	34
4.5	Investigation Into AlexNet QAT Results	37
5	Conclusions	41
6	Further Work	42
	References	43

1 Introduction

Deep Neural Networks(DNNs) have revolutionised computing in areas such as computer vision, voice recognition, and text classification. In the last decade, DNNs have proven to exceed the performance of existing techniques in many different types of application, which has led to a significant increase in their adoption, which can be attributed to the increased availability of computing power, adoption of cloud computing and greater availability of data.

Edge computing is the notion of bringing computation to the source of the data. In doing so, the application decreases its latency in response to new signals and can reduce the amount of bandwidth consumed. Many Edge applications are in environments with limited or no connectivity to the Internet, meaning computation cannot be off-loaded to an external server or cloud. Often there is no permanent energy source making batteries a requirement. An Edge device is one that has been designed to work under these constraints, and the devices themselves are typically resource-constrained so that their energy usage can be reduced. The compromise is a device with limited processing power and memory compared to a conventional Desktop computer or cloud server. Conversely many of the highest performing DNNs require a considerable amount of computing power and memory bandwidths [11].

The primary metric that has driven the design of DNNs has been accuracy and has meant that architectures have evolved without regard to computational complexity or energy usage, which makes the deployments onto an Edge device with a limited energy budget difficult [22]. Enabling DNNs to work on Edge devices can unlock new applications such as Federated Learning, allowing a user to gain the benefits of Neural Networks, whilst maintaining the control over their sensitive data, which is becoming an increasingly pressing issue.

1.1 Challenges of the Project

There are two main challenges in making a Deep Neural Network work under the constraints of an Edge device. The first is the physical size of the network in terms of bytes consumed in RAM. AlexNet [15] is an example of a well-known Convolutional Neural Network that has achieved acclaim in its performance. However, in terms of memory consumed in both RAM

and storage, it consumes approximately 225 MBs. Many Edge devices can count the amount of main memory available in Kilobytes and therefore do not have this storage space or main memory available. The second challenge is the processing requirements of running inference on the network with an acceptable latency.

Many Edge applications are required to work in a real-time mode, and a high latency would make the application non-viable. It has been well established that many DNNs are over-parameterised and do not require the majority of their trainable parameters in order to maintain their test accuracy [9]. Latency can be thought of as proportional to the number of parameters and depth of the network. Reducing the amount of parameters in the network can, in theory, decrease the latency, and would mean a greater range of devices may make use of it.

1.2 Project Aims

In this project, we aim to research and utilise techniques that will enable the usage of DNNs on a variety of Edge devices. This includes implementing a version of the Lottery Hypothesis algorithm, which will be utilised for training a variety of well-known network architectures such as AlexNet on datasets such as MNIST and CIFAR-10. An additional aim is to apply a quantisation technique such as Quantisation Aware Training(QAT) to the same set of networks and datasets. We aim to explore the possible trade-offs made when minimising the network and look at the effects on performance characteristics such as test accuracy, latency and network size, looking at both techniques in isolation and in combination. An additional objective is to deploy these minimised networks onto examples of Edge devices in order to understand what further work would be required for a real-world application.

1.3 Paper Layout

The paper is organised as follows:

- Section 2 we outline the background material which was required to understand the experiments conducted.
- Section 3 we outline the datasets, DNNs and devices used for the experiments. We give an outline of the experimental method and a breakdown of key decisions made for the project.

- Section 4 we present the experimental results as well as a discussion around observed effects.
- Section 5 we give our concluding thoughts on the techniques used, and the experimental method.
- Section 6 we discuss further work which could be based on this project

2 Background Research

2.1 Overview

There has been much research directed towards techniques that can minimise DNNs. This includes designing architectures from scratch that do not require as many parameters and layers when compared to a conventional DNN such as MobileNetV2 [19], compression techniques making use of Huffman coding as in [8], pruning and quantisation. For this project, we focus our research on pruning and quantisation techniques as these are most often the go-to solution. The aim is to reduce the computational requirement and latency of DNNs whilst maintaining their accuracy. In the following sections, we give an overview of techniques utilised in the experimental section.

2.2 Quantisation

2.2.1 Overview

Quantisation is a technique for reducing the size and computational requirement of Neural Networks by converting some or all of the network parameters to a lower bit-width. Typically, this means converting the network’s tensor weights and activations from using floating-point 32-bit arithmetic to 8-bit integer numbers. The benefits of doing this include an increased speed of inference, a lower memory requirement, and a reduction in energy usage as there is less processing required. The disadvantage is the introduction of approximations into the network as there is less dynamic range (the range of representable numbers) which can lead to a lower test accuracy. Depending on the device on which the network is deployed, it can be a balancing act between an acceptable loss in accuracy and the benefits of reducing the memory and processing requirements at time of inference. In this section we discuss a selection of conventional quantisation techniques which have been utilised in the experimental sections.

Conversion

A 32-bit floating-point number as defined by [12] has a non-uniform dynamic range of $[-3.4e38, 3.40e38]$ and can represent approximately 4-billion numbers, an 8-bit unsigned integer (used in this project) has a uniform dynamic range of $[0, 255]$ which means that it can only represent 256 numbers. In order to convert a 32-bit floating-point number to an 8-bit unsigned integer a scaling factor is required.

$$x_{quantised} = Clip(Round(x_{float}/scale))$$

Where:

Round: function to convert rational numbers to integers based on a rounding-policy, for example $y = floor(x)$

Clip: function to remove outliers which fall out of the available range

$$scale = (2 * absolute_{max}/256)$$

The techniques described in the next section have different methods in order to find an appropriate scaling and clipping factor. An inappropriate selection of these parameters can have a dramatic effect on the network’s final accuracy.

2.2.2 Techniques

Dynamic Quantisation

Dynamic Quantisation is a simple technique, the weights of the network are converted to a selected precision ahead of time and are then kept fixed. The network’s activations are converted just before inference is computed. Fine-tuning of the quantisation is done at time of inference, a scaling factor is used to round a number to its nearest representation and is adjusted until an optimal value is found. Dynamic Quantisation is a simple technique but generally considered to be inefficient and can cause the network’s accuracy to drop significantly.

Static Quantisation

Unlike Dynamic Quantisation, both the weights and the activations are quantised ahead of time. There is added complexity as it requires post-training using a representative dataset, which is used during the quantisation process to measure a dynamic range which is then used to find accurate values for the scaling and clipping factors. Quantising ahead of time, in theory makes it faster than Dynamic as there is no need for conversion during inference, but requires additional post training steps. Dynamic and Static Quantisation can be categorised as post training techniques and are applied to trained networks.

2.2.3 QAT (Quantisation Aware Training)

As the name suggests QAT is done during the training of the network. Unlike post-training methods, QAT aims to reduce the losses introduced from quantisation by simulating the lower precision type during the forward, and backward passes of the network. This is done by rounding the data type to its nearest representation by the insertion of fake quantising and de-quantising layers into the network, the weights and activations are kept in their float-32 bit representation. The network is then trained, and is then converted post training. Shyam et al. (2021) describes a typical implementation of integer QAT quantisation of a tensor x as the following:

$$x_q = \min(q_{\max}, \max(q_{\min}, \lfloor x/s + z \rfloor))$$

Where:

$q_{\min}$ and $q_{\max}$: minimum and maximum representable values at a given bitwidth

s : signedness which is the scaling factor making x span $[q_{\min}, q_{\max}]$

z : the zero-point which allows for the real value of 0 to be representable in x_q

[23] describes a typical de-quantising process as follows:

$$\hat{x} = (x_q - z)s$$

The process introduces an approximation of the loss which would be present after conversion to real integer representations. The optimiser will now have an awareness and will have the opportunity to minimise the loss during training. In theory QAT can produce quantised networks with higher accuracies than the dynamic and static methods and does not require post training fine-tuning. The disadvantage is that training the network becomes more complicated as there are additional steps for injecting the fake quantising and de-quantising layers into the network. This may also require many more optimisation steps to optimise away from the error introduced.

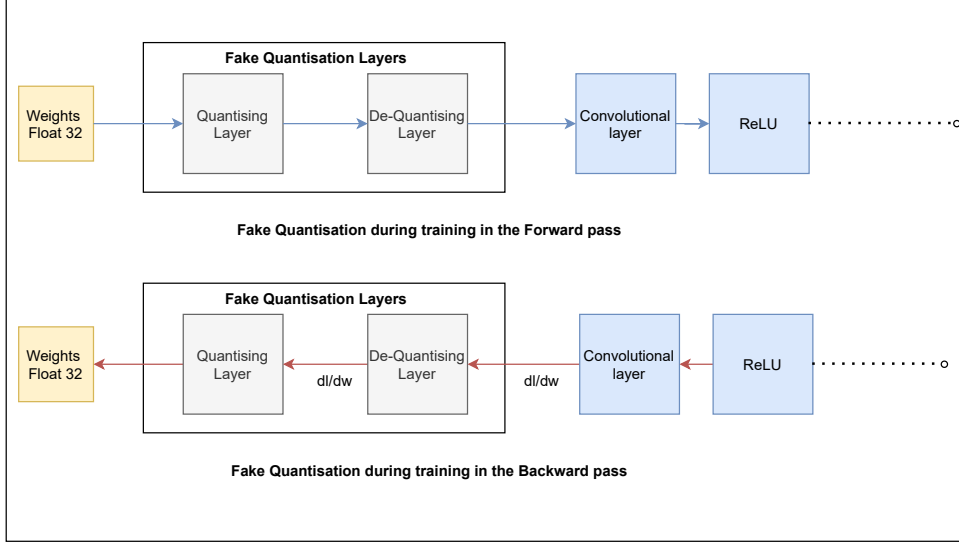


Figure 1: Example network showing the Forward and Backward During QAT

2.3 Pruning

Overview

Pruning Neural Networks is not a new area of research. It has been well established that most networks are over-parametrised, and many can be pruned to have a minority of their original parameters with minimal degradation in performance [9]. Aside from applications such as Edge devices, a reduction in a Neural Network’s parameters is desirable for properties such as speed of inference, a reduced memory bandwidth and can be used to reduce over-fitting [1].

In the last decade, there has been a need for deeper networks with many more trainable parameters in order to complete more complex tasks. Pruning has become an essential step in enabling the deployment of these networks onto constrained devices [22]. [9] sets out an influential pruning algorithm that many subsequent algorithms have been derived from, including the Lottery Hypothesis. The Lottery Hypothesis paper [4] outlines how a Dense network may contain a subnetwork that can be initialised in such a way that it can match the accuracy of the un-pruned network.

2.3.1 Techniques

Structured or Un-structured

Most pruning techniques can be categorised as structured or un-structured. Un-structured techniques generally prune by a set criterion, such as magnitude of the weights. When pruned a weight may be set to zero, removing the connected neurons’ effect on the rest of the network. The network’s structure is unchanged after pruning has been completed (see figure 2). Conversely, structured pruning has the effect of changing the network’s final structure by removing groups of weights such as an entire feature or filter.

Global or Local

Weights can be pruned globally or locally, local pruning prunes a percentage of weights at each layer. Global pruning prunes a percentage across the entire network.

Iterative or One-Shot

Most algorithms prune in either the one-shot mode or iteratively. The one-shot pruning process generally involves setting a desired amount of the network to prune and is done once, $p\%$ of weights are pruned. Iterative pruning is done over many pruning iterations, in each iteration $p^{1/n}\%$ of the weights are pruned. One-shot pruning can introduce a large amount of instability into the trained network, a technique which can train in the one-shot mode and maintain a good accuracy is a desirable goal for training efficiency.

2.3.2 The Lottery Ticket Hypothesis (LTH)

The recent 2019 paper, The Lottery Ticket Hypothesis [4] proposes that for many DNNs there exists an optimal sub-network at time of initialisation that can contain over 90% fewer parameters than the original network and when trained in isolation has a minimal loss in accuracy. The key problem up to this point has been that once pruned, a network cannot be effectively trained from scratch, WTS demonstrates that the initialisation of the network is the key to making this possible. From the empirical evidence presented in the paper, it can be speculated that many fully trained DNNs are over-parameterised, one of the reasons why they can generalise well is due to the

increased likelihood of the network containing one or more of these optimal subnetworks trained with a Winning Ticket.

Winning Ticket Search (WTS) Algorithm

The Winning Ticket Search algorithm is presented as a way of finding one of these optimal subnetworks that can be initialised in a way such that there is little degradation in performance (see algorithm 1). Frankle et al. presents two versions of WTS, one-shot and iterative, in this project we focus on the iterative algorithm. We wished to match the results of [4] as much as possible; the paper demonstrates winning tickets that match the accuracy of the original network at smaller sizes were found using the iterative method.

In the original LTH paper, the weights are pruned when their absolute value is below a set threshold and are then zeroed out with a mask. Deconstructing Lottery Tickets: Zeros, Signs, and the Supermasks [26] explores different threshold algorithms and shows some empirical success at improving the original WTS algorithm when applied to certain datasets and networks. A result of interest is the discovery of Supermasks, which are masks that can produce above chance performance. It was found that when applied to randomly initialised, untrained networks, networks with performance significantly better than chance were produced. The paper concludes that the sign of the weights of the reinitialised network is all that is needed in order to make the reinitialised network train.

A noted shortcoming of the WTS algorithm is that it has been shown not to scale well on larger more complex datasets, producing networks with poor test accuracy and significant training times. Frankle et al. speculates that this is due to the increased instability introduced from the complexity of the datasets. As noted by Zhou et al., the reasons why the LT algorithm works are not well understood, but perhaps gives a deeper insight into the internal mechanisms of DNNs.

Algorithm 1: The Winning Ticket Search algorithm [4]

- 1 Randomly initialise the network, setting the parameters to θ_0
 - 2 Train for i iterations, until the network converges, arriving at parameters θ_j .
 - 3 Prune $p\%$ of the parameters in θ_j layer-wise, using a threshold criterion and create a mask m .
 - 4 Reset the remaining parameters to their values in θ_0
 - 5 Mask the weights that have been pruned and freeze out during any subsequent training
 - 6 Repeat *Steps 2 to 5*, till max pruning iterations is reached $\theta_j \leq \theta_J$
 - 7 Create the winning ticket by applying the mask m to the original network $f(x; m \odot \theta_0)$ where $\odot$ is the element-wise dot-product.
-

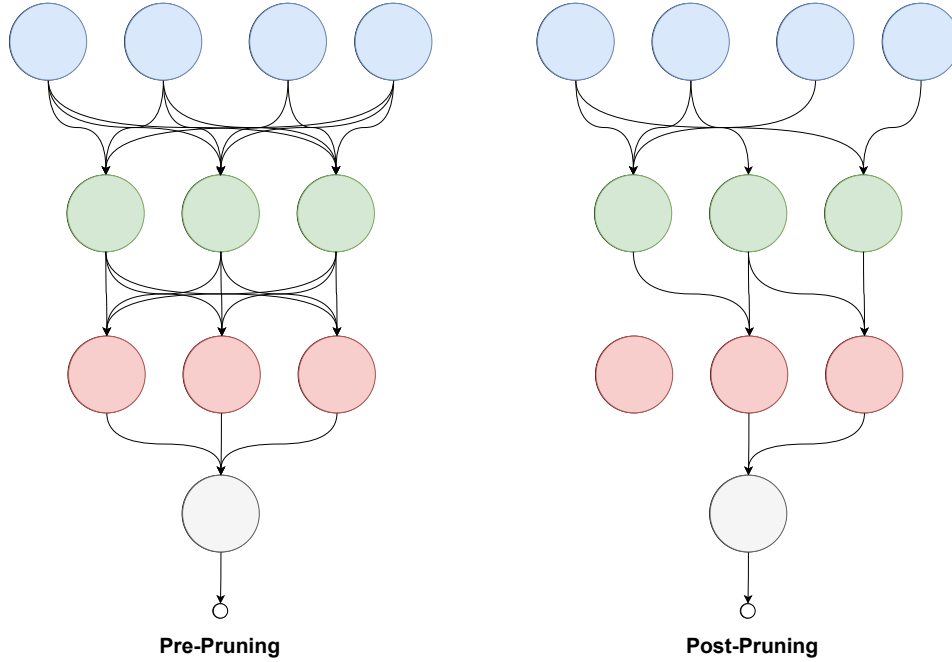


Figure 2: Un-structured Pruning: The contributions from connected neurons are removed

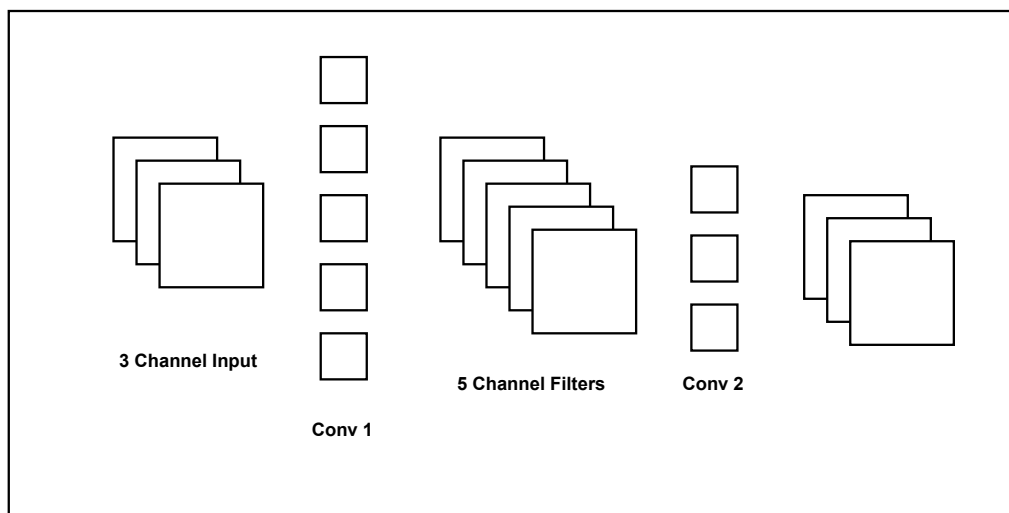


Figure 3: Structured Pruning - Example of a small convolutional layer before pruning

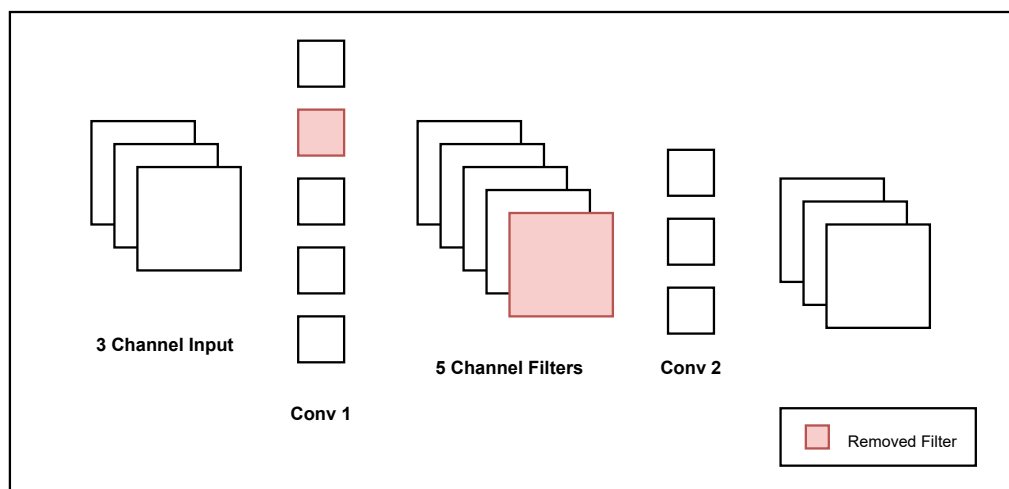


Figure 4: Structured Pruning - After pruning a filter has been removed

3 Experiments

3.1 Overview

The experiments aimed to investigate the effects of minimising the DNNs so that they can be deployed onto an Edge device. There were four objectives for the experiments:

- Explore and understand the impact of WTS pruning
- Explore and understand the impact of applying quantisation techniques in isolation
- Explore the impact of applying WTS and quantisation techniques in combination. To the best of our knowledge this is one of the first studies which look at the interplay between QAT and WTS
- Evaluate the resulting minimised networks by deploying and running inference on the selected Edge devices

Metrics Used

Depending on the application where the DNN is deployed, different metrics would be more critical; for example, if the application were to identify a suspected criminal from a high-resolution CCTV image, a high level of accuracy would be required for the network's legal usage. For other applications such as a consumer doorbell camera identifying if an image contains a human or an animal at the front door, the primary concern would be latency and false negatives would not be a critical concern. An Edge application designer would have these goals in mind, in this project we aimed to collect common metrics which could be used to evaluate these concerns. For each network and dataset we collected:

- **Test Accuracy** - The accuracy of the network after being fully trained
- **Test Loss** - The loss of the network after being fully trained
- **Network size** - The number of non-zero weights within the network
- **Speed of inference (Latency)** - The time for the network to run inference on a single sample

3.2 Datasets

Overview

The following datasets in Table 1 were selected as they represent common machine learning tasks and are often used as benchmarks.

Dataset	Dimensionality	Size	Training Set	Test Set
MNIST	784 (28×28 grayscale)	70k	60k	10k
CIFAR-10	1024 (32×32 colour)	70k	60k	10k

Table 1: Overview of Datasets used in experiments.

3.3 Networks

Dense Network

The Dense network has been created for this project and consists of three linear layers with 766,600 trainable weights, and is the least complex utilised in the experiments. It is not expected to yield high accuracy and low loss for the selected datasets but is used as a benchmark for the other networks. The network is also helpful in developing and testing code since the time to run is fast, and the number of trainable parameters is small in comparison to the other networks.

For MNIST the following transformations have been applied -

- Resize image to (28, 28),
- Normalise channels (0.5, 0.5)

The network is trained with a batch size of 512, and the number of workers has been set to 4.

AlexNet

AlexNet gained acclaim in 2012, where it achieved a winning top-5 test error rate of 15.3% in the ILSVRC-competition [15]. It is noted for its early adoption of GPUs for efficient computation and use of Dropout layers in order to reduce over-fitting. It is an eight-layer network with five convolutional layers and three linear layers. The Pytorch implementation used has approximately 160 million trainable weights. Since 2012 there have been many networks that have surpassed the performance of AlexNet,

but these mostly require many more layers making them much more computationally expensive. The implementation used is sourced from the Pytorch vision library [24]. The network has been adapted to work with MNIST and CIFAR-10.

For MNIST the following transformations have been applied -

- Grayscale conversion to 3 channels
- Resize to (224, 224)
- Normalise channels with a mean of 0.5

For CIFAR-10 the following transformations have been applied -

- Resize to 224
- We add noise with a center crop of 224
- Normalise with a mean of (0.485, 0.456, 0.406) and standard deviation of (0.229, 0.224, 0.225)

The network is trained with a batch size of 128, and the number of workers has been set to 2.

VGG-19

The VGG (Visual Geometry Group) [20] network is a successor of AlexNet. It primarily differs in the depth of the network, and the filter size used in the convolutional layers. Multiple configurations vary the number of layers of the network ranging from 11 to 19. The 19-layer network selected has approximately 220 million weights. The implementation used in the experimental section has been adapted from [16] which has been made to work with MNIST and CIFAR-10.

For MNIST the following transformations have been applied -

- Grayscale conversion to 3 channels
- Resize to (32, 32)
- Normalise channels with a mean of 0.5

For CIFAR-10 the following transformations have been applied -

- We randomly crop the image with a size of 32 and padding of 4

- We add noise by randomly flipping the image
- Normalise channels with a mean of (0.4914, 0.4822, 0.4465) and standard deviation of (0.247, 0.243, 0.261)

The network is trained with a batch size of 256, and the number of workers has been set to 4.

DLA

The DLA (Dense Layer Aggregation network) [25] aims to fuse information across layers by aggregation structures by merging feature hierarchies. The authors empirically show that DLA makes efficient use of fewer layers compared to other Convolutional Neural Networks including AlexNet. The network uses six layers and has approximately 243 million trainable weights. The implementation used in the experiments has been adapted from [16].

For MNIST the following transformations have been applied -

- Grayscale conversion to 3 channels
- Resize to (32, 32)
- Normalise channels with a mean of 0.5

For CIFAR-10 the following transformations have been applied -

- We randomly crop the image with a size of 32 and padding of 4
- We add noise by randomly flipping the image
- Normalise channels with a mean of (0.4914, 0.4822, 0.4465) and a standard deviation of (0.247, 0.243, 0.261)

The network is trained with a batch size of 64, and the number of workers has been set to 2.

3.4 Devices

Arm processors are preferred in Edge and mobile devices due to their low energy usage and cost over conventional X86 Intel or AMD CPUs [6]. Because of this, devices with ARM processors have been the focus of the project. We also include benchmarks for a CPU and a GPU as they give a relative comparison, and the majority of machine learning is done using conventional hardware.

Raspberry Pi 4: 4GB Model

The Raspberry Pi was initially released in 2012 and is one of the highest-selling single-board computers in history. Since its initial release, numerous iterations and model revisions have increased the unit's processing power, and the units are now comparable to a low-powered Desktop PC. The units are often deployed in Edge applications due to their low cost, energy usage and small form factor. The model selected for this project is the 4th revision with 4GB of RAM. It was selected due to its availability, clear documentation and popularity.



Figure 5: Raspberry Pi 4

Jetson Nano: 2GB and 4GB Models

The Jetson Nano shares many similarities with the Raspberry Pi. They both use an ARM processor, have a comparable hardware specification and share a compatible GPIO input. However, the Jetson Nano is an older generation device which makes use of a Quad-core ARM Cortex-A57 as opposed to the Raspberry Pi's newer generation Quad core Cortex-A72. In terms of raw computing power, the Raspberry Pi's A72 is approximately twice as

powerful when compared in CPU bound tasks. The main difference between the devices is the addition of a Nano module that contains CUDA cores typically found in Nvidia GPUs. The Jetson Nano has 128, and in theory this allows for a greater degree of parallelisation when compared to a typical multiple core CPU. The models used in this project are the 2GB and 4GB versions, which both share the same spec apart from additional RAM. The unit would be expected to have a higher performance characteristic when compared to the Raspberry Pi, due to its ability to offload computation to its CUDA cores.



Figure 6: Jetson Nano

Pytorch for Selected Devices

Although supported, there is currently no official build of Pytorch for ARM devices. This requires a build to be compiled from source on the target device (cross compilation is also possible) by the user. As we make use of quantisation in the project, we also need to compile the build with a quantisation backend. Currently, in Pytorch the only supported backends are FBGEMM [13] and QNNPACK [7]. FBGEMM is currently only supported on X86 architectures and QNNPACK is only supported on ARM.

3.5 Design and Implementation Choices

Lottery Hypothesis Winning Ticket implementation

The project focused on utilising the Lottery Hypothesis Winning Ticket algorithm; the first step was to identify existing implementations which could be used for the experiments. After a survey of open-source repositories, many were found to be of poor quality, which would not allow for the experiments to be done repeatably.

The notable exception was the original author’s implementation [3] which is of good quality and could have been used with minimal modification. It was felt that there would be a missed opportunity from not implementing the algorithm from scratch. Doing this would aid in understanding and allow a design that would enable the project experiments to be done scientifically and allow a greater degree of customisation.

We made use of [17] as an initial guide, particularly for the re-initialisation of the layers, but we extend and create a design around interfaces allowing for a variety of experiments to be conducted repeatably. A library for the implementation has been produced.

Language and Framework

The next decision was the language that would be used to write the implementation. Python is one of the most popular languages used today due to its pseudocode-like syntax and comprehensive range of libraries. Its wide adoption in machine learning made it the obvious choice for the project, others were not considered due to the limited timeline.

The last decision was the framework, the most popular Deep Learning frameworks in Python are Pytorch and Tensorflow. Although Pytorch is not as well established as Tensorflow, its usage is growing due to its more straightforward code interface and stability. Pytorch was primarily opted for due to its built-in support for quantisation techniques and documentation.

There was a noted risk that Pytorch might not support all the same features as Tensorflow, it was planned to mitigate this risk with the creation of scripts which could convert a network trained in Pytorch into a representation which could be utilised in Tensorflow and Tensorflow lite.

3.6 Experimental Method

3.7 Phase 1: Winning Ticket Search

Aim

The first set of experiments aimed to take the selected DNNs and datasets and apply the Winning Ticket algorithm. The aim is to find a Winning Ticket so that the redundant weights can be removed and a reduced network produced. We aim to observe the effects on the test accuracy and loss of the fully trained pruned networks as weight sparsity is increased. Weight sparsity is the number of weights with a zero magnitude, in the project implementation any weight with a magnitude less than 0.000001 is counted as a zero weight.

Method

The first iteration of the search is a conventional training loop, which trains the network to a pre-selected epoch using an optimiser (in this project we use Stochastic Gradient Descent (SGD)). The number of training epochs selected aims to reach an optimal test accuracy of the network on the selected dataset and is then kept fixed. The network is then pruned layer-wise by a percentage using a magnitude criterion creating a mask which zeros out a percentage of the weights. The non-zero weights are then reset to their initial value and the loop is repeated until the selected number of pruning iterations is reached.

For every pruning iteration, metrics for test accuracy and loss are recorded, and the state of the network is saved to a file every 5th pruning iteration. We wished to keep the experiments consistent by using the same optimiser with the same starting parameters, for each experiment we use SGD with a learning rate of 0.01, and a momentum of 0.9, these are typical values. We also make use of a cosine annealing learning rate scheduler which adjusts the learning rate throughout training this was found to improve test accuracy during initial testing.

Table 2 shows the WTS experiments with the hyper-parameters used, these values have been selected based on a trade-off between gaining the best accuracy possible and completing the experiment in a reasonable amount of time using the hardware at hand. The AlexNet, CIFAR-10 experiment took around 20 hours to complete, we wanted to keep this as an upper limit,

so we could test as many networks as possible in the project timeline. We based the number of training iterations on an initial loop and observed what accuracy could be obtained. We also conducted a series of initial experiments using the Dense network and found that a low prune % with a high number of iterations yielded smaller networks with higher accuracy.

Dataset	Network	Pruning Iterations	Training Iterations	Prune per iteration(%)
MNIST	Dense	40	15	5
MNIST	AlexNet	40	15	5
CIFAR-10	AlexNet	40	15	5
MNIST	DLA	30	5	10
MNIST	VGG-19	40	15	5
CIFAR-10	VGG-19	40	200	5

Table 2: Table of Winning Ticket search experiments and their parameters

3.8 Phase 2: Winning Ticket Search with QAT

Aim

The second phase of the experiments aimed to see the effect on test accuracy and loss by applying Quantisation Aware Training on the networks used in combination with WTS. We aimed to compare the non-quantised networks produced in phase 1 at the same amount of weight sparsity. QAT and WTS search are run in combination.

Method

QAT is required to be done during training, this meant that the networks would have to be re-trained with the additional quantising and de-quantising layers present. The easiest method found to insert the layers was to copy the network in code and add in the required layers manually, creating a new QAT network. Once done the same Winning Ticket interface is used. The number of training iterations is generally increased in order to optimise the network in the initial loop to an optimal test accuracy. This is necessary as the fake quantising layers add additional noise into the network. The network is converted into its quantised form before test accuracy and loss are recorded.

Note: Quantising is not currently supported on Nvidia GPUs in Pytorch, so these results cannot be included.

Dataset	Network	Pruning Iterations	Training Iterations	Prune per iteration(%)
MNIST	Dense	40	25	5
MNIST	AlexNet	40	40	5
MNIST	DLA	15	10	10
MNIST	VGG-19	40	15	5
CIFAR-10	VGG-19	40	200	5

Table 3: QAT Winning Ticket Experimental Parameters

3.9 Phase 3: Post Training Testing

Dynamic Quantisation on Pruned networks

Aim

The aim of this experiment is to apply Dynamic Quantisation in combination with WTS. Doing this would allow a comparison between non-quantised sparse networks, and the networks produced in phase 2. As Dynamic Quantisation is a post-training technique a fully trained network is required. To do this we aimed to use the sparse networks produced in phase 1, apply Dynamic quantisation and then record test accuracy and loss.

Latency Measurement on Devices

Aim

The aim of this experiment is to compare the effect of applying WTS search combined with Dynamic and QAT quantisation to the networks as weight sparsity is increased. We look at the latency of the networks.

Method

The code in listing 1 shows the method used for measuring latency on the devices under test.

```
def measure_model_latency(
    model: nn.Module,
    input_size,
```

```

device = torch.device("cpu")) -> np.array:

model.to(device)
model.eval()

# We use the torch utils benchamrk package to avoid
issues whilst benchmarking GPUs as this includes
explicit CUDA synchronisation calls and
## and warm up period
timer = torch.utils.benchmark.Timer(
    setup='from __main__ import __model__',
    globals={'x': (torch.rand(size=input_size).to(
        device)), 'model': model}
)

timing = timer.blocked_autorange(min_run_time=20).times
return np.array(timing)

```

Listing 1: Code to measure latency of the models

Issues

Measuring an accurate number for the latency of the networks proved to be a difficult task. Simple techniques such as taking the time before and after inference, then calculating the difference did not produce a stable result. It was thought that a statistical analysis would be possible by simply repeating the measurement thousands of times and allowing the measurement to trend towards its expected value; this still did not yield stable results with values varying wildly between runs.

The measurement issue has multiple contributing factors; the first is the concurrent nature of the operating system the network is running on. A background task may suddenly take processing priority at any time, causing the running inference task to be suspended which can cause a clock skew in the CPU and inaccurate measurement. The second issue stemmed from PyTorch’s multi-threaded set-up; it was found that using the default setting using all available threads, a repeatable timing was never observed. A third issue was found when attempting to measure using a GPU, due to the asynchronous loading of data and the synchronisation of calls between the CPU and GPU. These issues were exacerbated when using a Desktop with many more threads than the Edge devices.

To reduce these issues, the number of threads was reduced to no more than 8. The data loading issue was avoided by using a random sample generated on the device under test; using a random sample is still useful as it gives us a measure of the computational complexity of the network under test. We make use of the Pytorch benchmark utils package, which makes explicit CUDA synchronisation calls before and after timing and includes a warmup period as the initial loading takes longer than subsequent runs and would skew the results.

4 Results

4.1 Dense Network

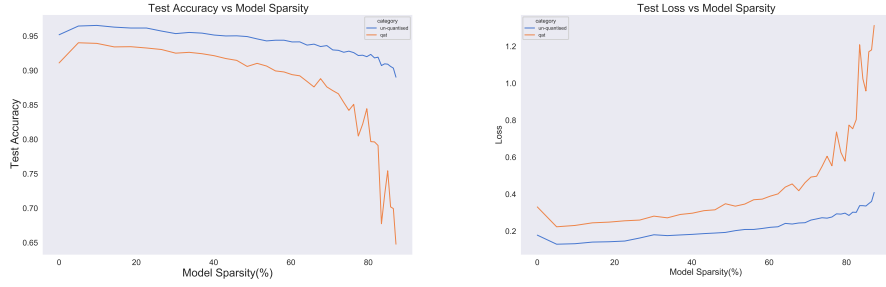


Figure 7: Training MNIST on Dense network using WTS QAT vs Non-quantised network

The graphs shown in figure 7 show the test accuracy and loss of the Dense network after being fully trained, using WTS as weight sparsity is increased. It can be observed that the un-quantised network maintains a good test accuracy up to 80% sparsity; however, we have stopped pruning too early and the maximum benefits of WTS search have not been reached. If pruning were to continue, we would likely observe a more significant drop in accuracy and increase in loss.

We can compare the result with WTS pruning applied to the Dense network combined with QAT. It can be observed that the conversion has not greatly impacted the network’s performance at low levels of sparsity. When the loss is increased past 60%, the network’s accuracy and loss become unstable; loss increases dramatically, and accuracy plummets and starts to perform much worse than the un-quantised network. For some Edge applications the drop in accuracy converting to the QAT network could be acceptable.

The graph shown in figure 9 shows the accuracy and loss of the Dense network measured in different states as weight sparsity is increased using WTS. The un-quantised network has no quantisation; the weights and activation are kept at float 32 bit precision. The Dynamic network is the conversion of the un-quantised network, post-training using Dynamic Quantisation, the weights are converted to 8-bit Integers. The QAT network has been trained with the same hyper-parameters but with the addition of

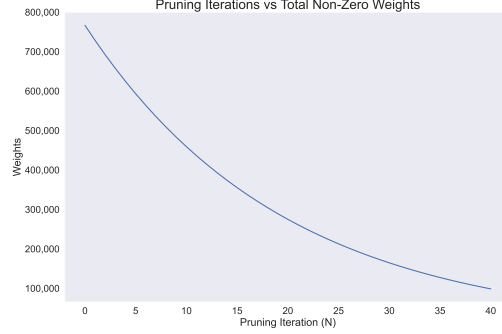


Figure 8: Reduction of weights with Pruning epoch

fake quantising and de-quantising layers. The network is then converted so that the weights and activations in the network are 8-bit integers.

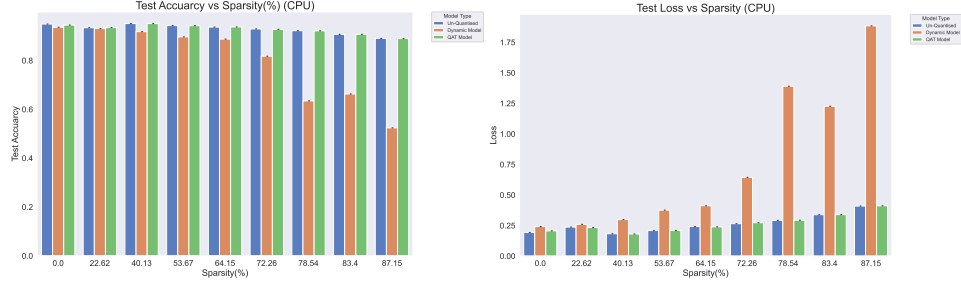


Figure 9: Accuracy and Loss compared with Quantisation and WTS Pruning

It can be observed that at the lower sparsity levels, all methods can reach a similar level of accuracy; loss is slightly increased in the dynamic and QAT networks. At the higher sparsity levels, the accuracy and loss of the Dynamic network are significantly affected. Accuracy is decreased by 40%, and loss increases dramatically. The QAT network can maintain the same level of accuracy and loss as the un-quantised network. The network's un-quantised size before quantisation is 1,052 KB; after quantisation the network is 283 KB.

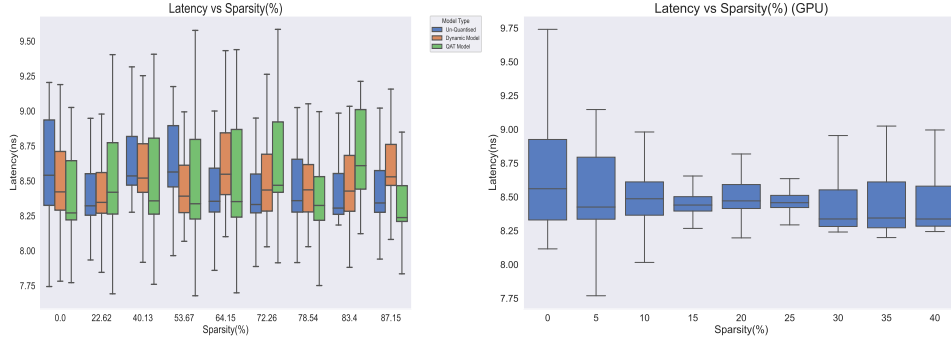


Figure 10: MNIST Inference Latency with Quantisation and WTS Pruning (CPU vs GPU)

Figure 10 shows the latency of the networks when running inference on a Desktop CPU and GPU. The error bars on the figure show the minimum and maximum values recorded and are the outliers. Firstly, looking at the CPU results, it can be said that Quantisation has had a small, but beneficial effect on the latency of the network, with QAT performing better than Dynamic. This falls in line with expectations as the conversion of the weights has been done ahead of time. The effect is likely small as the network’s starting complexity is low and there is not much gained from the optimisation.

Weight sparsity has a negligible impact on latency when used on both the CPU and GPU. We would expect the performance of the GPU to be better than that of the CPU. However, this is not observed, most likely due to the small size not taking full advantage of the available parallelism.

Figure 11 shows the same experiment, with the network deployed onto a Raspberry Pi, here we observe a strange result, with the un-quantised network performing better than the Dynamic and QAT networks. QAT performs the worst. This result does not make much sense and falls outside of our expectations. This indicates that there has been an issue converting the networks for the Raspberry Pi. The latency measurements are also not consistent.

Possibly this issue stems from converting the network trained using the FBGEMM quantisation backend and deploying it onto the Raspberry Pi,

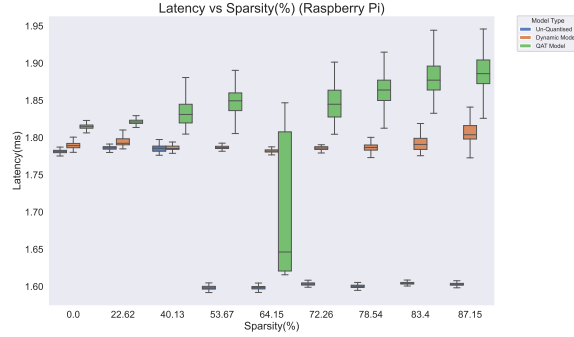


Figure 11: MNIST Inference Latency with Quantisation and WTS Pruning (Raspberry Pi CPU)

which uses QNNPACK. The parameters used to calculate the scaling factor are likely incorrect. In this case, we observe that WTS pruning decreases the un-quantised network’s latency.

Figure 12 shows the networks deployed onto the 2GB Jetson Nano, using the device’s CPU and GPU. In this case, we do not observe the same effect as seen on the Raspberry Pi. The Dynamic and QAT networks perform the same or slightly better than the un-quantised network. Again QAT does not perform as we would expect, having a higher latency than the Dynamic networks. We would also expect the latency to be higher on the Jetson Nano CPU compared to the Raspberry Pi due to its more powerful CPU, but this is not observed.

Using the device’s GPU, we now can observe the benefit from higher levels of concurrency. Comparing this to the Desktop CPU and GPU, due to the low amount of RAM available, the benefits of deploying onto the GPU become clear. WTS pruning has also had an impact on the latency of the networks.

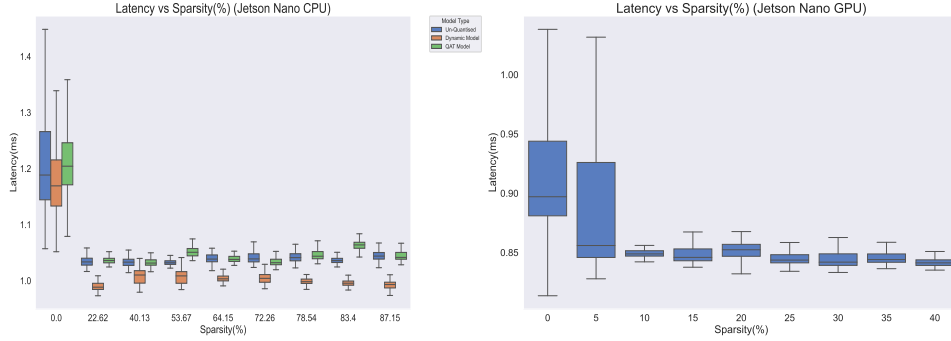


Figure 12: Dense Network MNIST Inference Latency Jetson Nano - CPU vs GPU

4.2 VGG-19

VGG-19 is considerably larger in terms of the number of trainable weights when compared to the Dense network tested; we would expect the network to be more resilient to pruning as there is a potential for a more significant amount of redundancy.

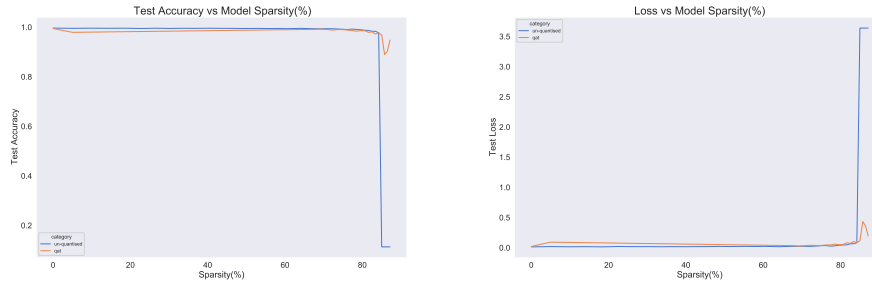


Figure 13: Training MNIST on VGG-19 network using WTS

Figure 13 shows the accuracy and loss of VGG-19 during the WTS process. As can be observed, the network performs exceptionally well. There is no degradation in accuracy or loss up to 87% sparsity. At this point, the network's accuracy suddenly nose dives and loss rises sharply; this may indicate that the minimum number of weights required for accurate prediction has been reached. Figure 14 shows the number of weights that have been removed during pruning, as was expected, we have been able to

remove a large amount. Compared to the network with WTS applied with QAT applied during training, it can be observed that the network has not been affected with no loss introduced and no decrease in accuracy. We only see a reduction at the highest sparsity level of 87% and have likely not seen the full benefits of applying WTS and more weights could be removed. A surprising result is that the network has performed better with QAT applied than the un-quantised network.

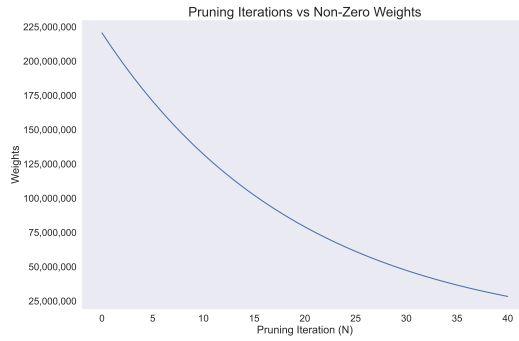


Figure 14: Reduction of weights with Pruning epoch

Figure 15 shows the network tested in different states on a deployed network resulting from the previous experiments. The Dynamic network would only be expected to have at most the same accuracy as the un-quantised network. VGG-19 with quantisation applied performs well, with Dynamic quantisation able to keep to the same level as the un-quantised version at all levels of sparsity. The network’s saved file size before quantisation is 78 MB; after quantisation this is reduced to 19.8 MB.

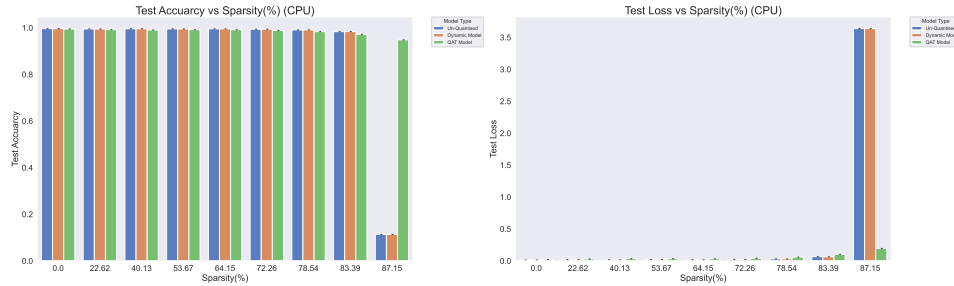


Figure 15: VGG-19 MNIST Accuracy and Loss compared

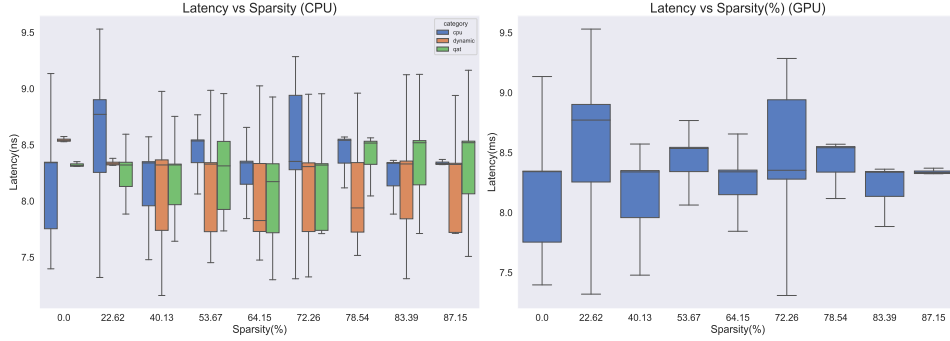


Figure 16: VGG-19 Inference Latency CPU vs GPU

Figure 16 shows the latency of the networks running inference on a Desktop CPU and GPU. No apparent effect on latency using either of the Quantisation techniques or WTS pruning can be observed. There appears to be no consistency in the measure of latency with pruning applied and at different levels of sparsity. At the very least we would expect to see a reduction in latency resulting from quantisation. Again, we do not observe a decrease in latency from running inference using the GPU. The GPU’s main benefits appear to be when running batches of data instead of a single sample.

Unfortunately, we faced multiple issues running the network on the Edge devices. Firstly, the network would not run on either the Jetson Nano 2GB or 4GB version’s GPU; after investigating, it appears that the memory of the devices is insufficient. We can run the network on the device’s CPU by using a swap file that gives extra memory capacity by utilising the device’s storage, this is not supported by the GPU.

A second issue faced was not being able to utilise the trained QAT networks with the device crashing with unsupported operations errors. This was later realised to be caused by a mistake in our method. The networks are trained using an Nvidia GPU, converted on the Desktop’s X86 processor, and then saved to a file. The networks should have been kept in their un-quantised form, saved to a file, and converted on the target device. Doing this would quantise the network with all the supported operations of the device. We could do this on the Dense network as all the operations were supported on

both architectures, as the network is simple. Fortunately, we have been able to run the un-quantised and dynamic networks using the devices' CPU.

Figure 17 shows the Jetson Nano 2GB and 4GB devices running inference with VGG-19. Comparing both, we do not see any benefit from using LTH pruning with latency remaining consistent across all sparsity levels. Running the network on the Jetson Nano with the extra 2GBs of RAM has not impacted the performance.

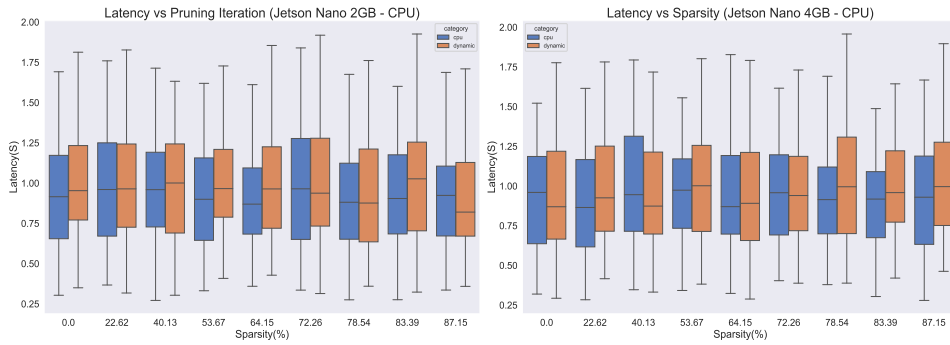


Figure 17: VGG-19 Inference Latency Jetson Nano 2GB vs 4GB

We believe that more RAM would be required to observe any benefit as the calls to the swap file are a bottleneck and are causing a significant slow-down. We believe this is also the reason why dynamic quantisation is running slower than the un-quantised network. The Raspberry Pi is running inference faster

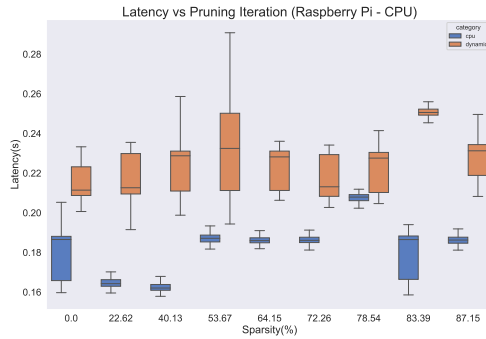


Figure 18: VGG-19 Inference Latency Raspberry Pi

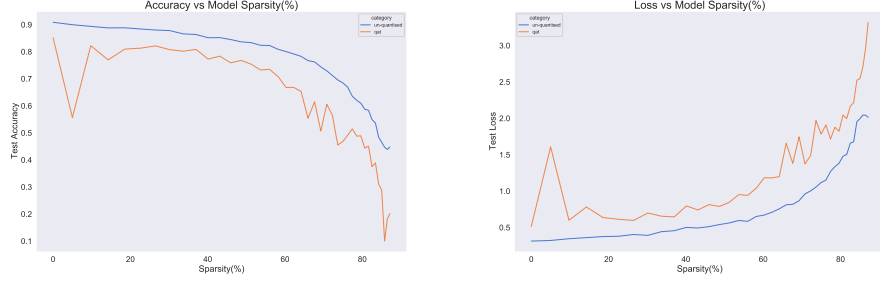


Figure 19: Training CIFAR-10 on VGG-19 network using WTS

and falls in line with expectations as the CPU is more powerful than the Jetson Nanos.

CIFAR-10

Figure 19 shows VGG-19 trained using CIFAR-10. CIFAR-10 is more complex than MNIST, and with 200 training epochs, the maximum accuracy reached was a reasonable 85% in the initial training loop.

As weight sparsity increases, we observe that WTS has not had the same effect as was observed with MNIST. As weight sparsity increases, the network cannot maintain the same levels of accuracy and loss. It might be possible to maintain a higher accuracy at higher sparsity levels if many more training iterations were done. Increasing the amount was not feasible on the available hardware in the given project timeline; in the LTH paper, the networks have been trained in the region of 50,000 training epochs and this might explain how they are able to obtain a higher test accuracy.

Comparing the accuracy of the quantised QAT network and the unquantised network, we can observe that the accuracy is close at all levels of sparsity; loss is slightly increased. QAT has not had a significant impact on the performance.

4.3 DLA

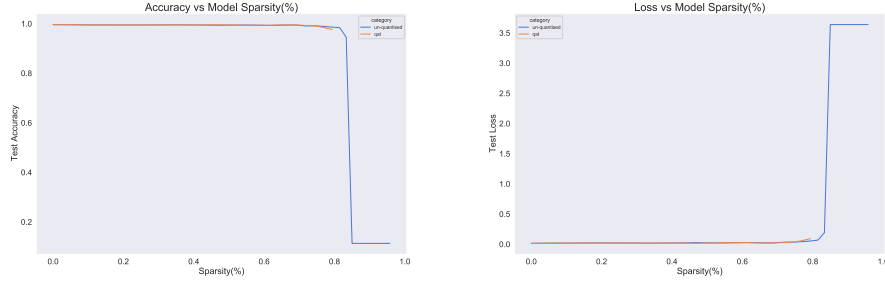


Figure 20: Training MNIST on DLA network using WTS

DLA is a large network with many layers and trainable weights; as observed in figure 20, the network has performed well when WTS pruning has been applied to trained using MNIST.

We can observe that up to 81% sparsity, the network maintains an accuracy above 98% with a minor loss. As was observed in VGG-19 and AlexNet, the minimum number of weights required for prediction has been reached. The network becomes too simple, and the network can no longer make accurate predictions; accuracy plummets, and loss rises sharply.

Comparing this to the QAT WTS experiment, it can be observed that the conversion to the quantised network has not significantly affected the accuracy of the network. We have not trained the QAT network to the same level of sparsity and would observe it follow the same pattern as the unquantised network.

Due to the issues faced with deploying the networks onto the devices, we did not have sufficient time to deploy DLA.

4.4 AlexNet

As was observed in the other networks, WTS pruning has performed well when trained using MNIST. Up to 87% of the weights have been removed with a slight decrease in accuracy and increase in loss. Pruning has been stopped too early, and the maximum benefits from using WTS have not been reached. There is a small amount of instability observed. Unlike the

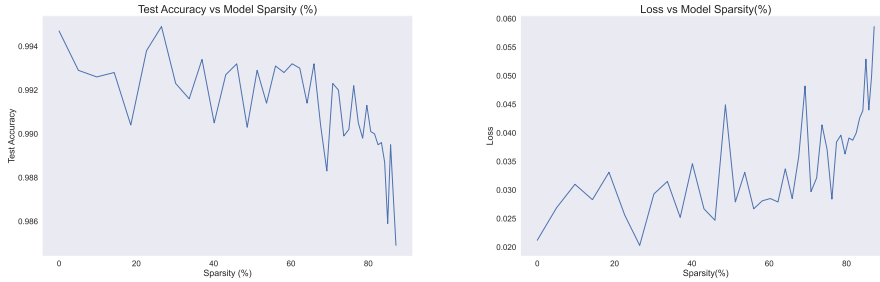


Figure 21: Training MNIST on AlexNet network using WTS

QAT experiments of the other networks, a strange result was observed, as can be seen in figure 22. After being trained, the test accuracy and loss of the network are unstable, dramatically going between a high and low test accuracy and loss as weight sparsity is increased. The experiment has been repeated multiple times, and the same result was always observed. See section 4.5 for further investigation on this issue.

Apart from the anomalous QAT result, the network performs well with dynamic quantisation applied, figure 24 shows there is almost no drop in accuracy and slight increase in loss as weight sparsity increases. AlexNet likely works well with WTS pruning as it has a large amount of redundancy due to its large number of weights. The file size of AlexNet before quantisation is 225 MB; after quantisation the file size is reduced to 59 MB.

In the case of AlexNet, it is challenging to say what the effect on latency has been. The results are un-stable, increasing and decreasing between quantisation and sparsity level. The project’s experiments have demonstrated that latency is an un-reliable measure for computational complexity. In both the CPU and GPU, WTS pruning has not had any observable effect.

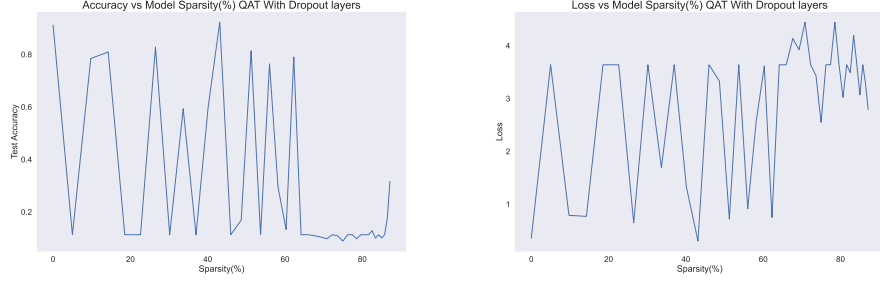


Figure 22: Training MNIST on AlexNet using QAT WTS

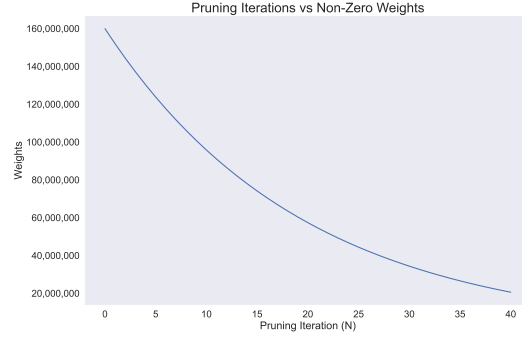


Figure 23: Reduction of weights with Pruning epoch

Figure 25 shows the latency of the model deployed onto the Desktop CPU and GPU. There is some indication that quantisation has decreased inference latency; however, the result is unstable, as has been observed in all other models. It is, therefore, difficult to say if pruning has had any effect on latency.

As observed in VGG-19, when WTS pruning has been applied to AlexNet trained on CIFAR-10, it has not been as effective and cannot maintain reasonable levels of accuracy and loss. We again speculate this is due to the number of training epochs; in this case, only 15 have been applied per pruning iteration compared to VGG-19.

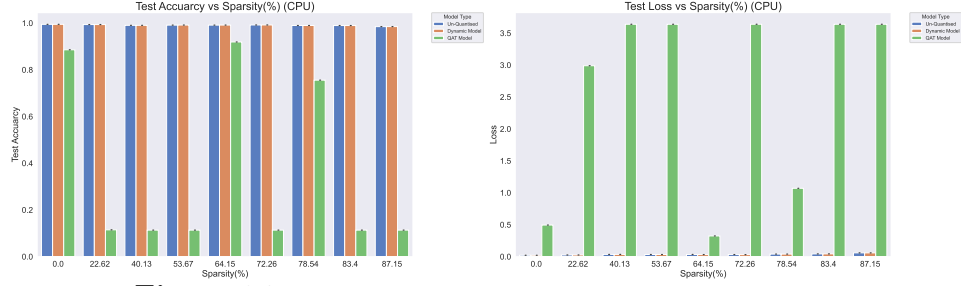


Figure 24: AlexNet MNIST Accuracy and Loss compared

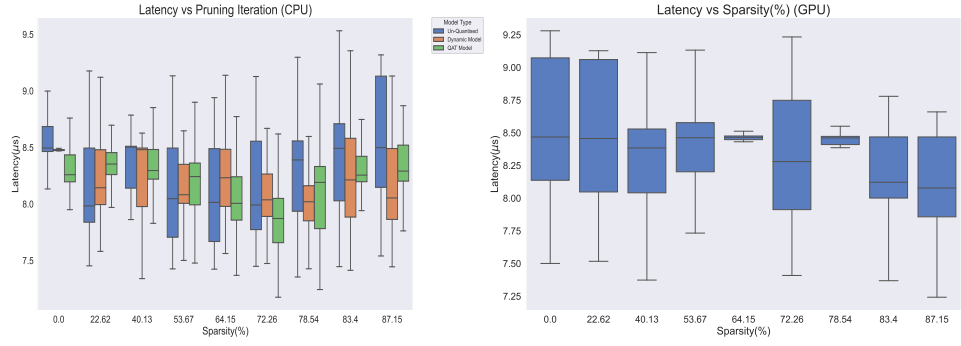


Figure 25: AlexNet MNIST Inference Latency CPU/GPU

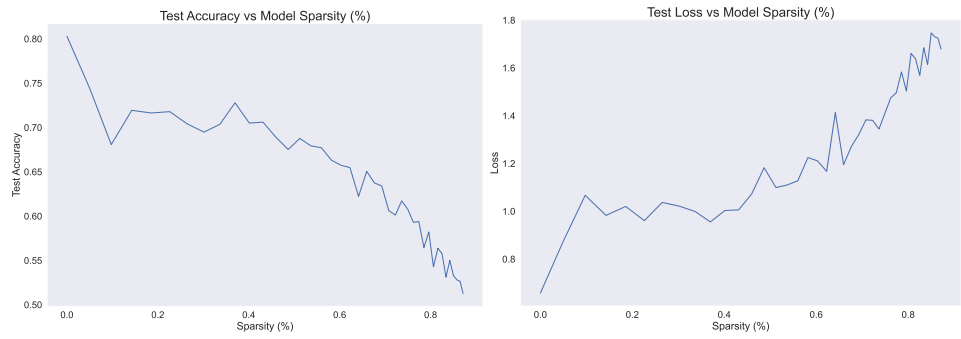


Figure 26: Training CIFAR-10 on AlexNet network using WTS

4.5 Investigation Into AlexNet QAT Results

Overview

The instability observed when applying QAT to AlexNet justified the need for a more in-depth investigation. The first step in doing this was to determine if there was an issue in the experimental setup. This was found to be identical to the other QAT experiments, with only the number of training and pruning iterations different. The same effect was observed when attempting to alter these values in both MNIST and CIFAR-10. This indicated that there was something about the underlying architecture of AlexNet which was causing the issue.

One layer which is not present in the other models was Dropout. Dropout is a popular regularisation technique introduced by Srivastava et al., [21] designed to reduce overfitting by randomly dropping units from the network during training. QAT is also applied during training, and the two techniques appear to be having some interaction which causes the observed instability.

Removing Dropout From AlexNet

A simple way to check if the Dropout layers are responsible is to remove them from AlexNet and repeat the QAT experiment with MNIST.

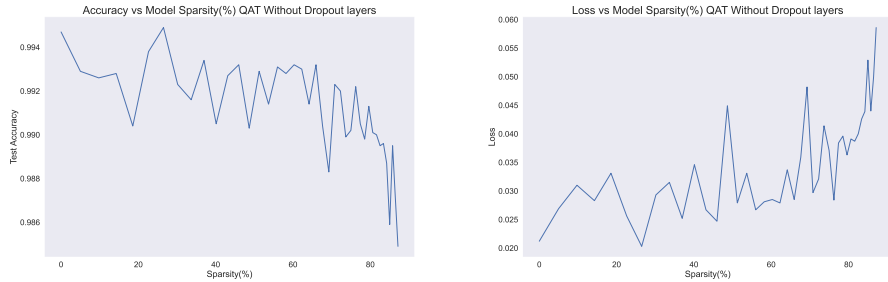


Figure 27: Training AlexNet QAT WTS with no Dropout layers

It can be observed in figure 27 that the instability in final test accuracy and loss is significantly reduced. The model now behaves in the same way as the other networks tested, with the accuracy improved and loss reduced. A small amount of instability still exists.

Applying Dropout to Dense Network

We next looked to see if the instability was isolated to Dropout or was some interaction with some other element within AlexNet. To do this, we make use of the Dense QAT Network with an additional Dropout layer.

The Dense network is a good candidate as it is simple, only being made up of three Linear layers and ReLU activation functions, there is no other form of regularisation applied. A single Dropout layer with a rate of 0.5 was inserted at the output of the 2nd layer, the same Dense QAT network experiment in phase 2 was repeated.

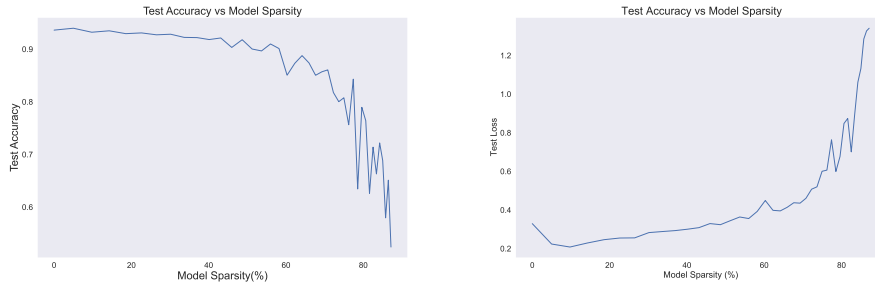


Figure 28: Training Dense Network with Additional Dropout Layer

It can be observed in figure 28 that the instability is not present. There is an improvement in the network's performance, as would normally be expected with the use of regularisation.

Increase the Amount of Dropout applied to Dense Network

We then looked to increase the amount of Dropout applied, increasing the rate to 0.95. After repeating the experiment, the effect started to appear. Test accuracy and loss between runs are now un-stable but not as pronounced as in AlexNet.

Suspected Cause

The effect seems to be due to an interaction between Dropout, QAT and WTS. We did not observe this when using Dynamic Quantisation, a post-training technique; we can reasonably conclude that this is introduced during training. Instability is not observed when a fair amount of Dropout is applied to the QAT Dense network but appears when increased too much.

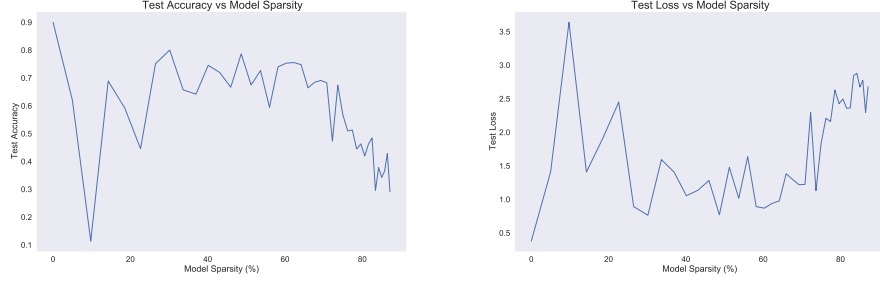


Figure 29: Training Dense Network with high amount of Dropout applied

Standard Dropout as described by [21] randomly sets a unit to zero with probability $1 - q$ for $0 \leq q \leq 1$. The activations are scaled by $1/q$ during training in order to preserve the mean. Han et al. (2015) describes a Dropout ratio adjustment for a pruned network in order to account for the change in the model capacity and recommends that the Dropout ratio be reduced quadratically with the number of active neurons.

We are likely observing the consequences of not accounting for this ratio adjustment. AlexNet, by default, has a large amount of Dropout applied but also has a large amount of redundancy due to the number of weights. When the non-QAT version is trained with WTS, we only observe a small amount of instability as the network can adapt to the un-adjusted Dropout ratio. However, with QAT and the introduction of the fake quantising and de-quantising layers during training, the complexity of the network is reduced so much it can no longer cope with the un-adjusted rate and the instability appears.

Possible Solution

Alpha Dropout, first described in [14], is designed for self-normalising networks. It aims to keep the input’s mean and variance to their original value after Dropout is applied. This property could be helpful for the pruned networks as it will automatically adjust the mean and variance which changes with network sparsity.

To test if using Alpha Dropout would be beneficial to use with QAT and LTH pruning, the QAT version of AlexNet was modified to replace the standard Dropout layers with Alpha Dropout at the same probability level.

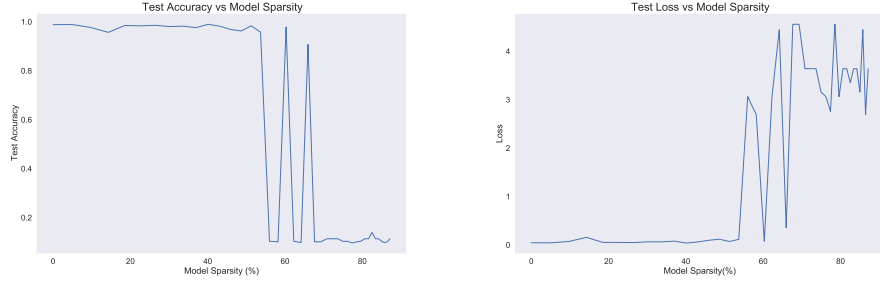


Figure 30: Training AlexNet with Alpha Dropout

The results observed in figure 30 show that the use of Alpha Dropout has been somewhat successful, there have been some clear benefits, noise is significantly reduced at moderate sparsity levels. However, past 53% sparsity, the instability reappears and acts the same as the network utilising standard Dropout.

The network has had too many weights removed, even with the adjustment in Dropout rate there is not enough redundancy, and the network is too simple to cope with quantisation. We would likely see more benefit from Alpha Dropout extended for greater levels of sparsity if the rate applied was significantly reduced. Alpha Dropout could be an effective method of using Dropout on pruned networks.

5 Conclusions

Referring back to the aims of the project, the results of the experiments have demonstrated that Edge devices are more than capable of utilising a variety of DNNs. For the chosen tasks, the networks could be run even without quantisation and/or pruning applied.

The second aim was to implement a version of LTH and utilise it on various networks to produce a minimised network and observe any effect this may have. This aim was accomplished, and we have shown the algorithm’s effectiveness on the selected networks empirically. We were able to prune up to 87% of the trainable weights and maintain a good test accuracy with a minor loss. However, when CIFAR-10 was trained, we found the algorithm’s effectiveness was reduced due to the added complexity of the dataset. Quantisation techniques, Dynamic and QAT, have proven to be the most effective tools for reducing computational complexity and the networks’ physical file size; typically, there was a 4x reduction.

Dynamic Quantisation is a simple technique that can easily reduce the size of a network, meaning more devices may use it but can cause a significant decrease in accuracy and increase in loss as weight sparsity increases. In comparison, QAT produced quantised networks with a smaller physical size and higher test accuracy maintained at a high level of weight sparsity. When utilised on certain networks, we found that QAT and WTS worked well in combination, observing minimal error introduced whilst gaining the benefits of a minimised network.

The final aim of the project was to deploy the minimised networks onto examples of Edge devices; this aim was partially met, we were not able to deploy all networks tested with LTH and QAT. We found the tooling selected for training and deploying the QAT networks to be poorly documented and complicated. There is no clear documentation on converting a trained QAT network using one backend onto a device using another. Larger networks such as AlexNet and VGG-19 proved to be the most difficult to deploy; often, the leading blocker was the amount of memory available on the device.

In most cases, the sparse networks did not directly translate into an improvement in inference latency. We speculate that this depends on the hardware architecture utilised; in some examples, we observed ARM processors gaining the most of the benefit of sparsity. Conventional

hardware does not seem to be set up to take advantage.

The networks tested have not been designed with Edge applications in mind; even with techniques such as pruning and quantisation, a designer choosing which network to utilise for an application should start with ones designed for constrained devices.

The technique utilised in this project to measure latency did not produce meaningful results. The larger, more complex networks gave wildly different values between runs, using it as a measure made it challenging to observe effects introduced from quantisation and weight sparsity. We would expect at the very least to see a reduction in latency when quantisation was applied, but this was not always observed. Latency, as a measure, should not be discounted, but a refinement of the method as used in the project is needed. Alternative metrics which do not depend on the activity of the CPU could be easier to utilise.

Refinement of the WTS algorithm is needed to reduce the training time required to find Winning Tickets, further work since the original publication should be explored with this aim in mind. Currently, it is too costly and requires considerable resources. Combining WTS with QAT produced interesting results, which should be investigated further.

6 Further Work

The results gathered from the project, particularly those from using WTS and QAT combined, have been good. However, it can be argued that both MNIST and CIFAR-10 are not complex datasets for modern DNN architectures to solve, even after being pruned and quantised. Given a larger hardware budget, we would investigate the effects on more complex ones such as ImageNet and other types of classification tasks such as text and voice.

We demonstrated that DNNs could be run on examples of Edge devices; the next step would be to use more constrained ones, designed to run off small batteries and with significantly less memory. The networks used in the project experiments had many trainable parameters giving a great deal of redundancy. We would investigate mobile architecture designed with fewer starting parameters and observe the effects of pruning and quantisation.

Numerous improvements could be made to the experimental method; these have been highlighted in the results section. We would look to improve the pipeline for converting the QAT networks to be deployed more easily on the Edge devices.

As highlighted, the technique for measuring latency in the project was found not to provide meaningful results. We would investigate and possibly extend alternative measures such as BOPs described by Karbachevsky et al. [2], which looks to count the number of bit-operations performed within the network, this could produce more stable results as it is not dependent on the activity of the CPU.

We would also like to explore and utilise some of the further work which has been done on the WTS algorithm since the release of the original paper, such as [[26], [5],[18]].

Finally, although numerous studies look at the interplay between pruning and quantisation, such as [8], to the best of our knowledge, there are few that study Quantisation Aware Training combined with Winning Ticket Search, we have only found a single study. [10] was only recently published in parallel to our work on this project; it focuses on a different type of data and only utilises a single network. However, it finds a similar result; QAT combined with pruning techniques such as LTH are complementary. The results from this project show that it could be an interesting area to examine. We observed in some cases that the networks were unaffected when both significantly pruned and quantised. The main benefits observed were a reduction in file size whilst maintaining high accuracy and low loss levels, many emerging Edge devices such as TPUs can only use integers. Therefore, techniques such as QAT are essential; these devices might be able to better exploit the benefits of the pruned networks.

References

- [1] Brian R. Bartoldson, Ari S. Morcos, Adrian Barbu, and Gordon Erlebacher. The generalization-stability tradeoff in neural network pruning, 2020.

- [2] Chaim Baskin, Natan Liss, Eli Schwartz, Evgenii Zheltonozhskii, Raja Giryes, Alex M. Bronstein, and Avi Mendelson. Uniq. *ACM Transactions on Computer Systems*, 37(1-4):1–15, Jun 2021.
- [3] Frankle. open lth. https://github.com/facebookresearch/open_lth. GIT.
- [4] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *ICLR*. OpenReview.net, 2019.
- [5] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel Roy, and Michael Carbin. Pruning neural networks at initialization: Why are we missing the mark? In *International Conference on Learning Representations*, 2021.
- [6] Kyriakos Georgiou, Zbigniew Chamski, Kris Nikov, and Kerstin Eder. A comprehensive and accurate energy model for arm’s cortex-m0 processor, 2021.
- [7] Qingchang Han, Yongmin Hu, Fengwei Yu, Hailong Yang, Bing Liu, Peng Hu, Ruihao Gong, Yanfei Wang, Rui Wang, Zhongzhi Luan, and Depei Qian. Extremely low-bit convolution optimization for quantized neural network on modern computer architectures. In *49th International Conference on Parallel Processing - ICPP*, ICPP ’20, New York, NY, USA, 2020. Association for Computing Machinery.
- [8] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding, 2016.
- [9] Song Han, Jeff Pool, John Tran, and William Dally. Learning both weights and connections for efficient neural network. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015.
- [10] Benjamin Hawks, Javier Duarte, Nicholas J. Fraser, Alessandro Pappalardo, Nhan Tran, and Yaman Umuroglu. Ps and qs: Quantization-aware pruning for efficient low latency neural network inference. *Frontiers in Artificial Intelligence*, 4:94, 2021.

- [11] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. Gpipe: Efficient training of giant neural networks using pipeline parallelism, 2019.
- [12] Institute of Electrical and Electronics Engineers. Ieee standard for binary floating-point arithmetic. IEEE Std 754-1985, 1985.
- [13] Daya Khudia, Jianyu Huang, Protonu Basu, Summer Deng, Haixin Liu, Jongsoo Park, and Mikhail Smelyanskiy. Fbgemm: Enabling high-performance low-precision deep learning inference. *arXiv preprint arXiv:2101.05615*, 2021.
- [14] Günter Klambauer, Thomas Unterthiner, Andreas Mayr, and Sepp Hochreiter. Self-normalizing neural networks, 2017.
- [15] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc., 2012.
- [16] kuangliu (Github username). pytorch vgg-13/19 and simple dla implmentation. <https://github.com/kuangliu/pytorch-cifar/blob/master/models>. Git hash:49b7aa9.
- [17] rahulvigneswaran. Lottery-ticket-hypothesis-in-pytorch. <https://github.com/rahulvigneswaran/Lottery-Ticket-Hypothesis-in-Pytorch>. GIT.
- [18] Alex Renda, Jonathan Frankle, and Michael Carbin. Comparing rewinding and fine-tuning in neural network pruning. In *International Conference on Learning Representations*, 2020.
- [19] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Mobilenetv2: Inverted residuals and linear bottlenecks, 2019.
- [20] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015.

- [21] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.
- [22] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel Emer. Efficient processing of deep neural networks: A tutorial and survey, 2017.
- [23] Shyam A. Tailor, Javier Fernandez-Marques, and Nicholas D. Lane. Degree-quant: Quantization-aware training for graph neural networks, 2021.
- [24] Pytorch alexnet implmentation. https://pytorch.org/hub/pytorch_vision_alexnet/. Accessed: 09/08/2021.
- [25] Fisher Yu, Dequan Wang, Evan Shelhamer, and Trevor Darrell. Deep layer aggregation, 2019.
- [26] Hattie Zhou, Janice Lan, Rosanne Liu, and Jason Yosinski. Deconstructing lottery tickets: Zeros, signs, and the supermask. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.